Xi_EE_Pose

A practical, branch-by-branch refactoring guide

Separate hand-eye calibration and reprojection while extracting shared parsing, transform, plotting, camera, video, matching, and I/O functionality.

Repository	C:\Users\Tracy\Projects\Xi_EE_Pose
Starting branch	main at b426d57 when this guide was prepared
Working branch	refactor/separate-workflows
Primary workflows	hand-eye calibration; reprojection
Approach	test behavior, extract shared code, migrate one caller at a time

How to use this guide

- Complete one phase at a time and run its checkpoint before committing.
- Type the commands from the repository root in PowerShell.
- Keep numerical behavior unchanged during structural commits.
- Check off the completion lists. Stop and investigate if a checkpoint fails.

Important: the repository currently has a modified .gitignore and an untracked requirements.txt. Preserve those changes before creating the branch.

1. Architecture and boundaries

The target is a Python package with two workflow namespaces supported by small, reusable common modules. The existing scripts remain available until replacements have been validated.

```
Xi_EE_Pose/
|-- pyproject.toml
|-- README.md
|-- src/xi_ee_pose/
|   |-- common/
|   |   |-- config.py       # dataclasses and workflow settings
|   |   |-- models.py       # typed observations and results
|   |   |-- parsing.py      # strings/records -> NumPy arrays
|   |   |-- transforms.py   # pure rigid-transform mathematics
|   |   |-- camera.py       # intrinsics, checkerboard, projection
|   |   |-- matching.py     # timestamps and frame association
|   |   |-- io.py           # CSV, NPZ, MAT loading and saving
|   |   |-- plotting.py     # reusable Matplotlib helpers
|   |   `-- video.py        # OpenCV reading, writing, overlays
|   |-- hand_eye/
|   |   |-- data_preparation.py
|   |   |-- checkerboard.py
|   |   |-- motions.py
|   |   |-- solver.py
|   |   |-- diagnostics.py
|   |   |-- results.py
|   |   `-- cli.py
|   `-- reprojection/
|       |-- chains.py
|       |-- projection.py
|       |-- errors.py
|       |-- overlays.py
|       |-- trajectories.py
|       `-- cli.py
|-- scripts/                # thin compatibility entry points only
|-- tests/
|-- data/
|-- outputs/
`-- archive/legacy_scripts/
```

Dependency direction

```
CLI modules  ---> workflow modules ---> common modules
                                    ---> NumPy / pandas / OpenCV / Matplotlib

common modules must never import hand_eye or reprojection.
hand_eye and reprojection should not import each other's CLI modules.
```

Definition of a library function

- Receives values or paths explicitly instead of reading module globals.
- Does not execute merely because its module was imported.
- Returns data or a figure instead of deciding how the user sees it.
- Does not mix numerical calculation with CSV writing or plotting.
- Uses frame-bearing names such as camera_T_board, not unexplained T1.

2. Phase 0 - protect the current work

> **Outcome:** A clean and recoverable starting point.

Inspect the current changes. An untracked file is not shown by ordinary git diff, so inspect requirements.txt directly.

```
cd C:\Users\Tracy\Projects\Xi_EE_Pose
git status --short --branch
git diff -- .gitignore
Get-Content requirements.txt
```

If both files are intentional, commit them on main before branching:

```
git add .gitignore requirements.txt
git diff --cached
git commit -m "chore: define Python dependencies and ignored files"
```

If they are unfinished, stash them instead:

```
git stash push --include-untracked -m "WIP dependency setup before refactor"
git stash list
```

Create and publish the branch:

```
git switch -c refactor/separate-workflows
git push -u origin refactor/separate-workflows
git status --short --branch
```

If you stashed changes and they belong on the refactor branch:

```
git stash pop
git status --short
```

Checkpoint

[ ] git branch --show-current prints refactor/separate-workflows.

[ ] git status shows only changes you understand.

[ ] The branch exists on GitHub after the push.

Recovery commands

```
# Discard only an uncommitted file you explicitly choose:
git restore path/to/file.py

# Undo the latest local commit but retain its edits:
git reset --soft HEAD~1

# Move back to main without modifying it:
git switch main
```

> Never use git reset --hard as a routine learning shortcut; it can destroy local work.

3. Phase 1 - record the behavioral baseline

> **Outcome:** Known numerical outputs to compare after each migration.

Create docs/current_workflows.md. Record the exact command, inputs, outputs, units, and coordinate-frame meaning for one Xi and one Si run.

```
# docs/current_workflows.md

## Coordinate convention
- Transform notation: destination_T_source
- Homogeneous transform shape: (4, 4)
- Translation unit: meters
- Pixel order: (u, v)
- Checkerboard square size: _____ meters

## Hand-eye baseline
- Command: _____
- Dataset: _____
- Frames used: _____
- R_X: _____
- t_X: _____
- Mean residual: _____

## Reprojection baseline
- Command: _____
- Calibration file: _____
- Frames used: _____
- Mean pixel error: _____
- Output files: _____
```

Save a machine-readable baseline in tests/fixtures/baseline_metrics.json:

```
{
  "hand_eye": {
    "translation_m": [0.0, 0.0, 0.0],
    "mean_rotation_error_deg": 0.0
  },
  "reprojection": {
    "mean_pixel_error": 0.0,
    "frame_count": 0
  }
}
```

Replace the zero placeholders with observed values. The initial tolerances should be tight because structural refactoring should not change the algorithm.

[ ] One complete hand-eye command is documented.

[ ] One complete reprojection command is documented.

[ ] Frame conventions and units are explicit.

[ ] Representative numeric outputs are saved.

```
git add docs/current_workflows.md tests/fixtures/baseline_metrics.json
git commit -m "docs: capture calibration and reprojection baselines"
```

4. Phase 2 - create an installable package

> **Outcome:** Imports work from src/ and the old scripts are still present.

Create the directory structure and empty __init__.py files. In PowerShell:

```
New-Item -ItemType Directory -Force `
  src/xi_ee_pose/common, `
  src/xi_ee_pose/hand_eye, `
  src/xi_ee_pose/reprojection, `
  tests/common, tests/hand_eye, tests/reprojection, tests/fixtures

New-Item -ItemType File -Force `
  src/xi_ee_pose/__init__.py, `
  src/xi_ee_pose/common/__init__.py, `
  src/xi_ee_pose/hand_eye/__init__.py, `
  src/xi_ee_pose/reprojection/__init__.py
```

Add pyproject.toml:

```
[build-system]
requires = ["setuptools>=69"]
build-backend = "setuptools.build_meta"

[project]
name = "xi-ee-pose"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
  "matplotlib",
  "numpy",
  "opencv-python",
  "pandas",
  "scipy",
]

[project.optional-dependencies]
dev = ["pytest", "pytest-cov"]

[project.scripts]
xi-hand-eye = "xi_ee_pose.hand_eye.cli:main"
xi-reproject = "xi_ee_pose.reprojection.cli:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Install in editable mode:

```
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
python -c "import xi_ee_pose; print(xi_ee_pose.__file__)"
pytest
```

[ ] The import path points inside src/xi_ee_pose.

[ ] pytest starts successfully, even if it reports no tests yet.

[ ] Existing scripts and data have not been moved.

```
git add pyproject.toml src tests
git commit -m "build: add src package and pytest layout"
```

5. Phase 3 - extract transform parsing

> **Outcome:** Every supported serialized transform layout has one tested parser.

Start with explicit layouts. Do not guess whether a sequence is translation-first or row-major; ambiguity here can produce plausible but incorrect calibration results.

Create src/xi_ee_pose/common/parsing.py:

```
from __future__ import annotations

import re
from typing import Literal
import numpy as np

TransformLayout = Literal["row_major_3x4", "translation_first"]


def parse_numbers(value: object) -> np.ndarray:
    if value is None:
        raise ValueError("Transform value is missing")
    numbers = re.findall(
        r"[-+]?(?:\d*\.\d+|\d+\.?)(?:[eE][-+]?\d+)?", str(value)
    )
    return np.asarray([float(number) for number in numbers], dtype=float)


def parse_transform(value: object, layout: TransformLayout) -> np.ndarray:
    values = parse_numbers(value)
    if values.size != 12:
        raise ValueError(f"Expected 12 transform values, got {values.size}")

    transform = np.eye(4, dtype=float)
    if layout == "row_major_3x4":
        transform[:3, :] = values.reshape(3, 4)
    elif layout == "translation_first":
        transform[:3, 3] = values[:3]
        transform[:3, :3] = values[3:].reshape(3, 3)
    else:
        raise ValueError(f"Unsupported transform layout: {layout}")
    return transform


def parse_translation(value: object) -> np.ndarray:
    values = parse_numbers(value)
    if values.size != 3:
        raise ValueError(f"Expected 3 translation values, got {values.size}")
    return values
```

Create tests/common/test_parsing.py:

```
import numpy as np
import pytest
from xi_ee_pose.common.parsing import parse_transform, parse_translation


def test_parse_row_major_3x4():
    value = "1 0 0 0.1 0 1 0 0.2 0 0 1 0.3"
    actual = parse_transform(value, "row_major_3x4")
    expected = np.eye(4)
    expected[:3, 3] = [0.1, 0.2, 0.3]
    np.testing.assert_allclose(actual, expected)


def test_parse_translation_first():
    value = "0.1 0.2 0.3 1 0 0 0 1 0 0 0 1"
    actual = parse_transform(value, "translation_first")
    expected = np.eye(4)
    expected[:3, 3] = [0.1, 0.2, 0.3]
    np.testing.assert_allclose(actual, expected)


def test_rejects_wrong_value_count():
    with pytest.raises(ValueError, match="Expected 12"):
        parse_transform("1 2 3", "row_major_3x4")


def test_parse_translation():
    np.testing.assert_allclose(parse_translation("[1, 2, 3]"), [1, 2, 3])
```

```
pytest tests/common/test_parsing.py -q
git add src/xi_ee_pose/common/parsing.py tests/common/test_parsing.py
git commit -m "refactor: centralize transform parsing"
```

6. Phase 4 - extract rigid-transform math

> **Outcome:** Transform operations are pure, named, and unit tested.

Create src/xi_ee_pose/common/transforms.py:

```
from __future__ import annotations
import numpy as np


def validate_transform(transform: np.ndarray, atol: float = 1e-6) -> None:
    transform = np.asarray(transform, dtype=float)
    if transform.shape != (4, 4):
        raise ValueError(f"Expected shape (4, 4), got {transform.shape}")
    if not np.all(np.isfinite(transform)):
        raise ValueError("Transform contains non-finite values")
    if not np.allclose(transform[3], [0, 0, 0, 1], atol=atol):
        raise ValueError("Invalid homogeneous bottom row")


def invert_transform(transform: np.ndarray) -> np.ndarray:
    validate_transform(transform)
    rotation = transform[:3, :3]
    translation = transform[:3, 3]
    inverse = np.eye(4)
    inverse[:3, :3] = rotation.T
    inverse[:3, 3] = -(rotation.T @ translation)
    return inverse


def compose_transforms(*transforms: np.ndarray) -> np.ndarray:
    result = np.eye(4)
    for transform in transforms:
        validate_transform(transform)
        result = result @ transform
    return result


def transform_points(transform: np.ndarray, points: np.ndarray) -> np.ndarray:
    validate_transform(transform)
    points = np.asarray(points, dtype=float)
    if points.ndim != 2 or points.shape[1] != 3:
        raise ValueError("Points must have shape (N, 3)")
    return (transform[:3, :3] @ points.T).T + transform[:3, 3]


def project_to_rotation(matrix: np.ndarray) -> np.ndarray:
    u, _, vt = np.linalg.svd(np.asarray(matrix, dtype=float))
    rotation = u @ vt
    if np.linalg.det(rotation) < 0:
        u[:, -1] *= -1
        rotation = u @ vt
    return rotation
```

Create tests/common/test_transforms.py:

```
import numpy as np
from xi_ee_pose.common.transforms import (
    invert_transform, project_to_rotation, transform_points,
)


def test_inverse_round_trip():
    transform = np.eye(4)
    transform[:3, 3] = [0.1, -0.2, 0.3]
    np.testing.assert_allclose(transform @ invert_transform(transform), np.eye(4))


def test_transform_points():
    transform = np.eye(4)
    transform[:3, 3] = [1, 2, 3]
    points = np.array([[0, 0, 0], [1, 1, 1]])
    np.testing.assert_allclose(
        transform_points(transform, points), [[1, 2, 3], [2, 3, 4]]
    )


def test_projected_rotation_is_proper():
    noisy = np.eye(3)
    noisy[0, 1] = 0.01
    rotation = project_to_rotation(noisy)
    np.testing.assert_allclose(rotation.T @ rotation, np.eye(3), atol=1e-12)
    np.testing.assert_allclose(np.linalg.det(rotation), 1.0, atol=1e-12)
```

```
pytest tests/common/test_transforms.py -q
git add src/xi_ee_pose/common/transforms.py tests/common/test_transforms.py
git commit -m "refactor: extract rigid transform operations"
```

7. Phase 5 - model configuration and observations

> **Outcome:** Functions receive explicit inputs instead of machine-specific globals.

Create src/xi_ee_pose/common/config.py:

```
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True)
class CheckerboardConfig:
    columns: int
    rows: int
    square_size_m: float


@dataclass(frozen=True)
class CameraConfig:
    intrinsics_path: Path
    distortion_path: Path | None = None


@dataclass(frozen=True)
class HandEyeConfig:
    matched_transforms_csv: Path
    image_directory: Path
    camera: CameraConfig
    checkerboard: CheckerboardConfig
    output_directory: Path
    transform_layout: str


@dataclass(frozen=True)
class ReprojectionConfig:
    hand_eye_path: Path
    transforms_csv: Path
    video_path: Path | None
    camera: CameraConfig
    output_directory: Path
```

Create src/xi_ee_pose/common/models.py:

```
from dataclasses import dataclass
import numpy as np


@dataclass(frozen=True)
class PoseObservation:
    frame_id: int
    timestamp: float | None
    base_T_gripper: np.ndarray
    camera_T_target: np.ndarray


@dataclass(frozen=True)
class HandEyeResult:
    gripper_T_camera: np.ndarray
    translation_rmse_m: float
    rotation_rmse_deg: float
    frame_ids: tuple[int, ...]
```

Replace hard-coded paths gradually. First add argparse options to one script; do not rewrite every script in this commit.

```
from argparse import ArgumentParser
from pathlib import Path


def parse_args():
    parser = ArgumentParser()
    parser.add_argument("--matched-transforms", type=Path, required=True)
    parser.add_argument("--images", type=Path, required=True)
    parser.add_argument("--intrinsics", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    return parser.parse_args()
```

Search for remaining machine-specific paths after every migration:

```
rg -n "C:\\Users|D:\\" src scripts -g "*.py"
```

> Hard-coded paths may remain temporarily in legacy scripts, but none should exist in src/.

8. Phase 6 - separate import from execution

> **Outcome:** Importing any package module performs no I/O and opens no windows.

Convert each script into three layers: argument parsing, a callable run function, and a guarded main function.

```
def run(config: HandEyeConfig) -> HandEyeResult:
    observations = load_observations(config)
    result = solve_hand_eye(observations)
    save_result(result, config.output_directory)
    return result


def main() -> None:
    args = parse_args()
    config = config_from_args(args)
    result = run(config)
    print(f"Translation RMSE: {result.translation_rmse_m:.6f} m")


if __name__ == "__main__":
    main()
```

Code that must move out of module scope

- pd.read_csv, np.load, scipy.io.loadmat, and cv2.imread calls.
- VideoCapture and VideoWriter construction.
- Directory creation and output writing.
- Loops over frames or checkerboard images.
- plt.show, cv2.imshow, and GUI callbacks.
- Loading calibration files into global variables.

Quick import-side-effect test:

```
python -c "import xi_ee_pose.common.parsing"
python -c "import xi_ee_pose.hand_eye.solver"
python -c "import xi_ee_pose.reprojection.projection"
```

Each command should exit without reading data, creating files, or opening a window.

9. Phase 7 - consolidate hand-eye calibration

> **Outcome:** One hand-eye pipeline handles Xi and Si observations through adapters.

Use this pipeline boundary:

```
raw CSV / images
    -> data_preparation.load_*_observations()
    -> checkerboard.estimate_target_poses()
    -> motions.build_relative_motions()
    -> solver.solve_hand_eye()
    -> diagnostics.calculate_residuals()
    -> results.save_result()
```

Move functions in this order

- hand_eye_calib.py: solve_Rx, solve_tx, solve_X into hand_eye/solver.py.
- hand_eye_calib.py: residual_errors into hand_eye/diagnostics.py.
- solve_hand_eye_0612.py: build_relative_motions into hand_eye/motions.py.
- xi_hand_eye_calibration_0616.py: pose and A/B construction into data_preparation.py and motions.py.
- clean_si_api.py: checkerboard detection into hand_eye/checkerboard.py.
- NPZ/CSV serialization into hand_eye/results.py.

Synthetic AX=XB test

Create a known X and several non-degenerate B motions, calculate A = X B X^-1, then assert that the solver recovers X. Include rotations about different axes; translation-only or single-axis motion is degenerate.

```
def test_solver_recovers_known_transform():
    expected_x = make_known_transform()
    b_motions = make_non_degenerate_motions()
    a_motions = [expected_x @ b @ invert_transform(expected_x)
                 for b in b_motions]

    actual_x = solve_hand_eye(a_motions, b_motions)
    np.testing.assert_allclose(actual_x, expected_x, atol=1e-7)
```

Hand-eye CLI shape

```
xi-hand-eye   --matched-transforms data/xi_data_0616/matched_transforms.csv   --images
checkerboard_images/xi_20260616   --intrinsics C:/path/to/K_left.mat   --layout
translation_first   --checkerboard-columns 9   --checkerboard-rows 6   --square-size-m 0.005
 --output-dir outputs/hand_eye/xi_0616
```

> Use the actual checkerboard dimensions and square size from your experiment; the values above are examples, not repository facts.

10. Phase 8 - consolidate reprojection

> **Outcome:** One projection core supports images, plots, CSV errors, and video overlays.

Create src/xi_ee_pose/reprojection/projection.py:

```
from __future__ import annotations
import cv2
import numpy as np


def project_camera_points(
    points_camera: np.ndarray,
    intrinsics: np.ndarray,
    distortion: np.ndarray | None = None,
) -> np.ndarray:
    points_camera = np.asarray(points_camera, dtype=float)
    if points_camera.ndim != 2 or points_camera.shape[1] != 3:
        raise ValueError("points_camera must have shape (N, 3)")
    if np.any(points_camera[:, 2] <= 0):
        raise ValueError("All projected points must be in front of the camera")

    distortion = np.zeros(5) if distortion is None else distortion
    pixels, _ = cv2.projectPoints(
        points_camera,
        np.zeros(3),
        np.zeros(3),
        np.asarray(intrinsics, dtype=float),
        np.asarray(distortion, dtype=float),
    )
    return pixels.reshape(-1, 2)
```

Add a simple optical-axis test:

```
def test_optical_axis_projects_to_principal_point():
    intrinsics = np.array([[500, 0, 320], [0, 500, 240], [0, 0, 1]])
    pixels = project_camera_points(np.array([[0, 0, 1.0]]), intrinsics)
    np.testing.assert_allclose(pixels, [[320, 240]])
```

Make transform chains readable

```
camera_T_ee = camera_T_base @ base_T_ee
point_camera = transform_points(camera_T_ee, point_ee)
pixel = project_camera_points(point_camera, intrinsics)
```

Avoid unexplained chains such as A @ X @ inv(B). In calibration math A/B/X are conventional; in production reprojection code, frame-bearing names make direction errors visible.

Migrate in this order

- Pure 3D-to-2D projection and error calculations.
- Named coordinate-frame chain construction.
- Static checkerboard overlay.
- Per-frame error CSV generation.
- Video overlays and preview images.
- 3D Matplotlib diagnostics and trajectories.

11. Phase 9 - categorize plotting and video

> **Outcome:** Presentation code is reusable without contaminating numerical modules.

Create small helpers in common/plotting.py:

```
import numpy as np


def set_equal_3d_axes(ax, points: np.ndarray) -> None:
    points = np.asarray(points, dtype=float)
    minimum = points.min(axis=0)
    maximum = points.max(axis=0)
    center = (minimum + maximum) / 2
    radius = max((maximum - minimum).max() / 2, 1e-9)
    ax.set_xlim(center[0] - radius, center[0] + radius)
    ax.set_ylim(center[1] - radius, center[1] + radius)
    ax.set_zlim(center[2] - radius, center[2] + radius)
```

Plot functions return figures:

```
def plot_actual_vs_predicted(actual, predicted):
    figure = plt.figure(figsize=(9, 8))
    axis = figure.add_subplot(111, projection="3d")
    axis.scatter(*actual.T, label="Actual")
    axis.scatter(*predicted.T, label="Predicted")
    set_equal_3d_axes(axis, np.vstack([actual, predicted]))
    axis.legend()
    return figure


# The CLI decides presentation:
figure = plot_actual_vs_predicted(actual, predicted)
figure.savefig(output_path, dpi=160, bbox_inches="tight")
plt.close(figure)
```

Video helpers belong in common/video.py. Keep workflow-specific color meanings and labels in reprojection/overlays.py.

```
def draw_points(image, pixels, color=(0, 0, 255), radius=5):
    output = image.copy()
    for u, v in np.asarray(pixels):
        if np.isfinite(u) and np.isfinite(v):
            cv2.circle(output, (round(u), round(v)), radius, color, -1)
    return output
```

- [ ] Numerical modules do not import matplotlib.pyplot.
- [ ] Projection functions do not write images or videos.
- [ ] Plot functions return figures.
- [ ] Every VideoCapture and VideoWriter is released, including error paths.

12. Migrating one legacy script safely

> **Outcome:** A repeatable recipe for each dated script.

Apply the following loop to one script at a time.

Step 1 - classify every function

- Parsing -> common/parsing.py
- Rigid math -> common/transforms.py
- Camera math -> common/camera.py
- File formats -> common/io.py
- Calibration-specific -> hand_eye/
- Projection-specific -> reprojection/
- CLI and orchestration -> the corresponding cli.py

Step 2 - write a characterization test

Call the current function with a small deterministic input and capture its output. This is not an endorsement of the algorithm; it protects current behavior while moving it.

Step 3 - copy, test, redirect, then remove

```
# 1. Put the extracted function in src/.
# 2. Run the focused unit test.
pytest tests/common/test_transforms.py -q

# 3. Change one legacy caller to import it.
from xi_ee_pose.common.transforms import project_to_rotation

# 4. Run the legacy workflow and compare baseline output.
# 5. Remove only that duplicate implementation.
# 6. Commit immediately.
git add src tests scripts/the_migrated_script.py
git commit -m "refactor: use shared rotation projection in Xi calibration"
```

Step 4 - inspect the commit

```
git show --stat
git show --check
git status --short
```

> Do not combine function extraction, coordinate convention changes, unit conversion, algorithm replacement, and file renaming in the same commit.

13. Commit and validation checklist

> **Outcome:** Each commit is understandable and independently testable.

Recommended commit sequence

```
docs: capture calibration and reprojection baselines
build: add src package and pytest layout
test: cover known transform layouts
refactor: centralize transform parsing
refactor: extract rigid transform operations
refactor: extract camera projection helpers
refactor: add explicit workflow configuration
refactor: move hand-eye solver into package
refactor: add hand-eye command-line workflow
refactor: move reprojection core into package
refactor: add reprojection command-line workflow
refactor: centralize plotting and video helpers
docs: document package workflows and coordinate frames
chore: archive superseded scripts
```

Run before every commit

```
git status --short
git diff
pytest -q
git diff --check
```

Run at milestones

```
python -m xi_ee_pose.hand_eye.cli --help
python -m xi_ee_pose.reprojection.cli --help
pytest --cov=xi_ee_pose --cov-report=term-missing
rg -n "C:\\Users|D:\\" src -g "*.py"
```

Numerical comparison

- Same accepted/rejected frames.
- Same transform layout and units.
- R_X and t_X equal within chosen floating-point tolerance.
- Same projected pixel locations within tolerance.
- Same mean and per-frame error definitions.
- Expected output files are produced in the requested directory.

14. Archive legacy scripts only after verification

> **Outcome:** The active surface is small without losing historical work.

Create archive/legacy_scripts/README.md with a replacement table:

```
| Legacy script | Status | Replacement |
|---|---|---|
| xi_hand_eye_calibration_0616.py | archived | xi-hand-eye ... |
| reproject_Si.py | archived | xi-reproject ... |
| Reproject_updated_si.py | archived | xi-reproject ... |
```

Move files with git mv so Git can recognize renames:

```
New-Item -ItemType Directory -Force archive/legacy_scripts
git mv scripts/old_script.py archive/legacy_scripts/old_script.py
git status --short
pytest -q
git commit -m "chore: archive superseded workflow scripts"
```

Do not archive a script until its replacement has produced equivalent results on a representative dataset. scrap.py may be archived earlier if you explicitly establish that it is not part of an active workflow.

Data and generated outputs

- Keep small, intentional test fixtures in tests/fixtures/.
- Keep reproducible input data separate from generated outputs.
- Write new generated artifacts beneath outputs/.
- Avoid adding large videos or generated previews to new commits.
- Before removing tracked data, verify whether collaborators depend on it.

15. Pull request and merge workflow

> **Outcome:** A reviewable branch that can be merged without losing main.

Update the branch before opening the pull request:

```
git switch refactor/separate-workflows
git fetch origin
git merge origin/main
pytest -q
git push
```

For a first refactor, merge is easier to reason about than rebase because it does not rewrite your already-published branch commits.

Review the complete branch:

```
git log --oneline main..HEAD
git diff --stat main...HEAD
git diff main...HEAD
```

Pull request description template

```
## Purpose
Separate hand-eye calibration and reprojection into package workflows.

## Shared functionality extracted
- parsing
- rigid transforms
- camera projection
- plotting/video helpers
- CSV/NPZ/MAT I/O

## Validation
- [ ] pytest passes
- [ ] Xi baseline matches
- [ ] Si baseline matches
- [ ] hand-eye CLI smoke-tested
- [ ] reprojection CLI smoke-tested

## Behavior changes
None intended. List any known numerical differences here.

## Legacy code
List scripts retained or archived and their replacements.
```

After review and successful checks, merge through GitHub. Then update locally:

```
git switch main
git pull --ff-only origin main
git branch -d refactor/separate-workflows
```

16. Troubleshooting reference

> **Outcome:** Common refactoring failures have a contained response.

Imports fail with ModuleNotFoundError

```
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
python -c "import xi_ee_pose; print(xi_ee_pose.__file__)"
```

A script works directly but fails as a module

Replace imports such as from clean_si_api import ... with package imports such as from xi_ee_pose.common.camera import .... Run entry points through the installed package.

Calibration changed after moving code

- Compare parsed matrices before comparing final X.
- Check translation-first versus row-major layout.
- Check meters versus millimeters.
- Check destination_T_source direction and inverses.
- Check frame matching and order.
- Check that rotation projection occurs at the same stage.

Reprojection is mirrored, shifted, or off-screen

- Verify camera_T_point rather than point_T_camera.
- Verify positive camera-frame Z before projection.
- Verify intrinsics correspond to the image resolution and crop.
- Verify (u, v) is not treated as (row, column).
- Verify distortion coefficients and active ROI handling.

A commit contains too many unrelated changes

```
# Keep changes but unstage them, then stage coherent pieces:
git restore --staged .
git add -p
git diff --cached
git commit -m "refactor: describe one coherent change"
```

Need to compare with main at any point

```
git diff main...HEAD --stat
git log --oneline main..HEAD
```

17. Final completion checklist

[ ] Hand-eye calibration has a documented CLI entry point.

[ ] Reprojection has a documented CLI entry point.

[ ] Shared parsing has one implementation per supported data format.

[ ] Shared transform math has focused unit tests.

[ ] Importing package modules performs no I/O or GUI work.

[ ] No reusable src/ module contains a machine-specific absolute path.

[ ] Coordinate-frame direction and units are documented.

[ ] Synthetic AX=XB and camera projection tests pass.

[ ] Representative Xi and Si outputs match the recorded baseline.

[ ] Plot functions return figures; CLIs decide whether to save or show them.

[ ] Legacy scripts have a documented replacement or remain intentionally active.

[ ] README installation and usage commands work in a clean environment.

[ ] The pull request explains validation and any behavioral differences.

Suggested first milestone

Stop after the package skeleton, parsing, transform math, tests, and migration of one hand-eye caller plus one reprojection caller. Open a small pull request or ask for a review at that point. This proves the architecture before moving the largest scripts.

Guiding principle

Preserve behavior first, move code second, improve algorithms third. Calibration code can produce plausible results even when a frame direction, layout, or unit is wrong, so explicit conventions and small numerical tests are the foundation of this refactor.

Prepared for the Xi_EE_Pose repository on 2026-07-14.